At this point you're probably wondering how you can use these functions. That's my favorite part. Once you define the functions, the framework can expose them in different ways depending on your needs. For example, Spring Cloud Function can automatically expose the functions defined in `spring.cloud.function.definition` as REST endpoints. Then you can directly package the application, deploy it on a FaaS platform like Knative, and voilà: you've got your first serverless Spring Boot application. That's what we'll do in chapter 16 when we build serverless applications. Or you can use one of the adapters provided by the framework to package the application and deploy it on AWS Lambda, Azure Functions, or Google Cloud Functions. Or you can combine it with Spring Cloud Stream and bind the function to message channels in an event broker like RabbitMQ or Kafka.

Before we explore the integration with RabbitMQ using Spring Cloud Stream, I want to show you how to test the functions and their composition in isolation. Once the business logic is implemented as functions and tested, we can be sure it will work the same way, whether it's triggered by a REST endpoint or an event notification.

10.3.3 Writing integration tests with @FunctionalSpringBootTest

Using the functional programming paradigm, we can implement business logic in standard Java and write unit tests with JUnit without being affected by the framework. At that level there is no Spring code, just plain Java. Once you've ensured that each function works, you'll want to write some integration tests to verify your application's overall behavior when your functions are processed by Spring Cloud Function and exposed the way you configured.